

Quản lý thư viện

1. Singleton Pattern:

```
public class Library { 6 usages

    private static Library instance; 3 usages
    private List<Book> books; 3 usages

    private Library() { 1 usage
        books = new ArrayList<>();
    }

    public static Library getInstance() { 1 us
        if (instance == null) {
            instance = new Library();
        }
        return instance;
    }

    public void addBook(Book book) { 2 usages
        books.add(book);
    }

    public List<Book> getBooks() { 1 usage
        return books;
    }
}
```

Kết quả: Hệ thống chỉ tạo một đối tượng Library duy nhất và dùng trong suốt chương trình

```

public class Main {

    public static void main(String[] args) {

        // Singleton Pattern
        Library lib1 = Library.getInstance();
        Library lib2 = Library.getInstance();

        if (lib1 == lib2) {
            System.out.println("Both objects are the same instance");
        }
    }
}

```

Main x

```

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1
.1\lib\idea_rt.jar=56358:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8 -Dsun
.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\QLyThuVien\target\classes org.example.Main
Both objects are the same instance

Process finished with exit code 0

```

2. Factory Method Pattern:

```

public interface Book {
    void displayInfo();
    String getTitle();
    String getAuthor();
}

```

```

public class PaperBook implements Book { 1 usage

    private String title; 3 usages
    private String author; 2 usages
    private String category; 1 usage

    public PaperBook(String title, String author, String category) {
        this.title = title;
        this.author = author;
        this.category = category;
    }

    public void displayInfo() { 3 usages
        System.out.println("Paper Book: " + title);
    }

    public String getTitle() { return title; } 1 usage
    public String getAuthor() { return author; } 1 usage
}

```

Kết quả: Tạo các đối tượng Book khác nhau mà không cần khởi tạo trực tiếp class cụ thể

```

26 // Factory Method Pattern
27 Book book1 = BookFactory.createBook( type: "paper",
28     title: "Java Programming",
29     author: "James Gosling",
30     category: "Programming");
31
32 Book book2 = BookFactory.createBook( type: "ebook",
33     title: "Design Patterns",
34     author: "GoF",
35     category: "Software Engineering");
36
37 Book book3 = BookFactory.createBook( type: "audio",
38     title: "Clean Code",
39     author: "Robert C. Martin",
40     category: "Programming");
41
42 book1.displayInfo();
43 book2.displayInfo();
44 book3.displayInfo();
45
46

```

Run Main x

```

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1
.1\lib\idea_rt.jar=53924:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8 -Dsun
.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\QLyThuVien\target\classes org.example.Main
Paper Book: Java Programming
EBook: Design Patterns
Audio Book: Clean Code

```

3. Strategy Pattern:

```
public interface SearchStrategy { 3 usages 2 implementations
    List<Book> search(List<Book> books, String keyword);
}

public class SearchByAuthor implements SearchStrategy { no usages

    public List<Book> search(List<Book> books, String keyword) { no usages

        List<Book> result = new ArrayList<>();

        for (Book book : books) {
            if (book.getAuthor().toLowerCase().contains(keyword.toLowerCase()))
                result.add(book);
        }

        return result;
    }
}
```

Kết quả: strategy Pattern cho phép thay đổi cách tìm kiếm sách linh hoạt mà không làm thay đổi cấu trúc ban đầu

```
50
51 // Strategy Pattern:
52 List<Book> books = new ArrayList<>();
53
54 books.add(new PaperBook(title: "Java Programming", author: "James Gosling", category: "Programming"));
55 books.add(new EBook(title: "Design Patterns", author: "GoF", category: "Software"));
56 books.add(new AudioBook(title: "Clean Code", author: "Robert Martin", category: "Programming"));
57
58 // Tìm theo tiêu đề
59 SearchStrategy searchByTitle = new SearchByTitle();
60 List<Book> resultTitle = searchByTitle.search(books, keyword: "Java");
61
62 System.out.println("Search by Title:");
63 for (Book b : resultTitle) {
64     b.displayInfo();
65 }
66
67 // Tìm theo tác giả
68 SearchStrategy searchByAuthor = new SearchByAuthor();
69 List<Book> resultAuthor = searchByAuthor.search(books, keyword: "Martin");
70
71 System.out.println("\nSearch by Author:");
72 for (Book b : resultAuthor) {
73     b.displayInfo();
74 }
```

Run Main x

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1\lib\idea_rt.jar=64679:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\QLyThuVien\target\classes org.example.Main

Search by Title:
Paper Book: Java Programming

Search by Author:
Audio Book: Clean Code

4. Observer Pattern:

```
public interface Observer { 5 usages
    void update(String message); 1
}
```

```
public class LibraryUser implements Observer { no usages

    private String name; 2 usages

    public LibraryUser(String name) { no usages
        this.name = name;
    }

    public void update(String message) { 1 usage
        System.out.println(name + " received notification: " + message);
    }
}
```

```

public class NotificationSystem { no usages

    private List<Observer> observers = new ArrayList<>();

    public void addObserver(Observer o) { no usages
        observers.add(o);
    }

    public void removeObserver(Observer o) { no usages
        observers.remove(o);
    }

    public void notifyObservers(String message) { no usages

        for (Observer o : observers) {
            o.update(message);
        }
    }
}

```

Kết quả: Observer Pattern cho phép nhiều đối tượng theo dõi một đối tượng khác. Khi có sự kiện xảy ra (như thêm sách mới), hệ thống sẽ tự động gửi thông báo đến các đối tượng đã đăng ký.

```

79 //Observer Pattern
80
81 NotificationSystem notificationSystem = new NotificationSystem();
82
83 // Tạo người dùng đăng ký nhận thông báo
84 Observer user1 = new LibraryUser( name: "Alice");
85 Observer user2 = new LibraryUser( name: "Bob");
86 Observer user3 = new LibraryUser( name: "Charlie");
87
88 // Đăng ký theo dõi
89 notificationSystem.addObserver(user1);
90 notificationSystem.addObserver(user2);
91 notificationSystem.addObserver(user3);
92
93 // Thông báo khi có sách mới
94 notificationSystem.notifyObservers("New book added: Design Patterns");
95
96
97 }
98 }

```

run Main x

```

C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1
.1\lib\idea_rt.jar=52649:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8 -Dsun
.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\QLyThuVien\target\classes org.example.Main
Alice received notification: New book added: Design Patterns
Bob received notification: New book added: Design Patterns
Charlie received notification: New book added: Design Patterns

```

5. Decorator Pattern:

```
public interface Borrow { 7 us
    String getDescription();
}

public class BasicBorrow implements Borrow { 1 u

    public String getDescription() { 3 usages
        return "Borrow book";
    }
}

public abstract class BorrowDecorator implements Borrow {

    protected Borrow borrow; 4 usages

    public BorrowDecorator(Borrow borrow) { 2 usages
        this.borrow = borrow;
    }

    public String getDescription() { 3 usages 2 overrides
        return borrow.getDescription();
    }
}

public class ExtendTimeDecorator extends BorrowDecorator { 1 usage

    public ExtendTimeDecorator(Borrow borrow) { no usages
        super(borrow);
    }

    public String getDescription() { 3 usages
        return borrow.getDescription() + " + extended borrow time";
    }
}
```

Kết quả: Decorator Pattern cho phép mở rộng chức năng của đối tượng mà không cần thay đổi lớp gốc.

```
    // Decorator Pattern |
    Borrow borrow = new BasicBorrow();

    borrow = new ExtendTimeDecorator(borrow);
    borrow = new SpecialEditionDecorator(borrow);

    System.out.println(borrow.getDescription());
}
```

Main x

| 🖨️ 🔍 🔄 :

```
C:\Users\nguye\.jdk\ms-21.0.8\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1
.1\lib\idea_rt.jar=54168:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.1.1\bin" -Dfile.encoding=UTF-8 -Dsun
.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath T:\KTMT\QLyThuVien\target\classes org.example.Main
Borrow book + extended borrow time + special edition
```